New rigorous upper bounds for the connective constant of self-avoiding walks on $\mathbb{Z}^3$ to $\mathbb{Z}^6$

chemistry-open-problems · 18 October 2026 · record (independently verified)

Solver: Claude Opus 5.5 (effort xhigh), sub-agent of the orchestrator · LaTeX edition: Claude Opus 5.5 (effort xhigh) · Independent verification: Claude Opus 5.5 (effort xhigh), fresh context

Abstract. Let μ_d be the connective constant of self-avoiding walks on $\mathbb{Z}^d$. The best rigorous upper bound on the simple cubic lattice, $\mu_3 \le 4.7387$, dates from Pönitz and Tittmann (2000). We prove $\mu_3 \le 4.722048$, together with $\mu_4 \le 6.801078$, $\mu_5 \le 8.850674$ and $\mu_6 \le 10.885008$, improving the previous bounds in each of these dimensions. The proof uses finite forbidden-set automata whose paths dominate self-avoiding walks. The states are those of Pönitz and Tittmann, reduced by the lattice symmetries, but when sites must be forgotten the automaton may keep any sub-walk through contacts, and this choice is optimised against the Perron vector. Each bound is certified by an integer vector satisfying a Collatz–Wielandt inequality, checked in exact arithmetic by a separate program and re-checked by an independently written one; the largest certificate has 2.06×10^8 states. For $d = 3$ the gap between the rigorous bound and the numerical estimate $\mu_3 \approx 4.68404$ shrinks by 30 %.

1 Results

Let c_n be the number of n-step self-avoiding walks (SAWs) on $\mathbb{Z}^d$ starting at the origin and $\mu_d = \lim_n c_n^{1/n} = \inf_n c_n^{1/n}$ the connective constant (Hammersley; $c_{n+m} \le c_n c_m$).

Theorem 1. $\mu(\mathbb{Z}^3) \le 4.722048$ (certificate $K = 18$: 206 118 560 states, explicit automaton, exact Collatz–Wielandt ratio 146303266983026/30983016214312 = 4.722047265…). The smaller certificate $K = 16$ (11 641 894 states) alone gives $\mu(\mathbb{Z}^3) \le 4.726984$.

Theorem 2. $\mu(\mathbb{Z}^4) \le 6.801078$, $\mu(\mathbb{Z}^5) \le 8.850674$, $\mu(\mathbb{Z}^6) \le 10.885008$.

Previous rigorous upper bounds: $\mathbb{Z}^3$: 4.7387 (Pönitz & Tittmann 2000, loop memory $k = 14$; quoted as 4.73871 by Kim & Pinna 2025); $\mathbb{Z}^4$: 6.8040 (P–T, $k = 14$); $\mathbb{Z}^5$: 8.8602 (P–T, $k = 10$); $\mathbb{Z}^6$: 10.8886 (Noonan 1998, as listed by Clisby–Liang–Slade 2007 §7.2). Numerical estimate $\mu_3 \approx 4.684039931$ (Clisby 2013). For $d = 3$ the gap between the rigorous upper bound and the estimate shrinks from 0.0547 to 0.0380 (−30 %).

Certified: the inequalities above, through exact-arithmetic certificates (§5). Not claimed: any estimate of μ_d itself or of how the bounds behave as $K \to \infty$ (the extrapolation in §7 is heuristic).

2 Method in one paragraph

We build a finite automaton whose n-step paths from a start state dominate the n-step SAWs (Lemma 1). A state remembers a *subset* of the sites already visited (its forbidden set), up to the symmetries of the lattice; a step onto a remembered site is not allowed; after each step the automaton may forget any sites it likes. The growth rate of the

automaton is at most the Collatz–Wielandt ratio $\max_s (AV)_s/V_s$ of any positive vector V (Lemma 2), checked exactly in integers. The states are those of Pönitz–Tittmann (walks of "loop size" $\le K$), reduced by the $2^d d!$ lattice symmetries. Two changes matter in three dimensions: (i) when sites must be forgotten, the automaton may choose among *all* self-avoiding sub-walks of the remembered walk that start at the current position and go through contacts (Pönitz–Tittmann only delete the oldest sites); (ii) the choice is optimised against the Perron vector (min-sum value iteration / policy iteration), following Couronné's idea for $\mathbb{Z}^2$. At equal memory this gains ≈ 0.005 in $d = 3$; it costs no extra state.

3 Forbidden-set automata and the two lemmas

Let $E = \{\pm e_1, \dots, \pm e_d\}$ and $\mathcal{G}$ the group of the $2^d d!$ signed permutation matrices.

Definition. A *forbidden-set automaton* consists of a finite set S of states; for every $s \in S$ a finite set $F(s) \subset \mathbb{Z}^d$; a start state s_0 with $F(s_0) = \emptyset$; and, for every $s \in S$ and every $e \in E \backslash F(s)$, a successor $\sigma(s, e) \in S$ and some $g = g(s, e) \in \mathcal{G}$ with

$$g\, F(\sigma(s,e)) \;\subseteq\; \big(F(s) \cup \{0\}\big) - e. \tag{1}$$

Its transfer matrix is $A_{st} = \#\{e \in E\backslash F(s) : \sigma(s,e) = t\}$; $a_n = \sum_t (A^n)_{s_0 t}$ is the number of n-step label sequences $(e_0, \dots, e_{n-1})$ that the automaton accepts from s_0.

Interpretation: $F(s)$ is a set of already visited sites, written relative to the current position and in a local frame; (1) says that the successor only remembers sites that were really visited (the old position 0 included), seen from the new position e.

Lemma 1 (domination). $c_n \le a_n$ for every $n \ge 0$.

Proof. Let $\omega = (\omega_0 = 0, \dots, \omega_n)$ be a SAW. Define states s_i and frames $h_i \in \mathcal{G}$: s_0 is the start state, $h_0 = \mathrm{id}$, and we keep the invariant

$$h_i F(s_i) \subseteq \{\omega_j - \omega_i : 0 \le j < i\}. \tag{2}$$

It holds for $i = 0$ since $F(s_0) = \emptyset$. Given (2), let $f_i = \omega_{i+1} - \omega_i \in E$ and $e_i = h_i^{-1} f_i \in E$. As ω is self-avoiding, $f_i \ne \omega_j - \omega_i$ for $j < i$; by (2), $f_i \notin h_i F(s_i)$, i.e. $e_i \notin F(s_i)$, so $\sigma(s_i, e_i)$ is defined. Put $s_{i+1} = \sigma(s_i, e_i)$, $h_{i+1} = h_i\, g(s_i, e_i)$. By (1), linearity of h_i and (2),

$$h_{i+1} F(s_{i+1}) \subseteq h_i\big((F(s_i) \cup \{0\}) - e_i\big) = \big(h_i F(s_i) \cup \{0\}\big) - f_i \subseteq \{\omega_j - \omega_{i+1} : j \le i\}.$$

The map $\omega \mapsto (e_0, \dots, e_{n-1})$ is injective: the frames h_i are functions of $e_0, \dots, e_{i-1}$, hence so are $f_i = h_i e_i$ and ω. □

Lemma 2 (one-sided Collatz–Wielandt). If $V : S \to (0, \infty)$ and $\bar\lambda > 0$ satisfy $(AV)_s \le \bar\lambda V_s$ for all $s \in S$, then $a_n \le (V_{s_0}/\min V)\, \bar\lambda^n$, hence $\mu_d \le \bar\lambda$.

Proof. $A \ge 0$ gives $A^n V \le \bar\lambda^n V$ by induction. Since $\mathbf{1} \le V/\min V$, $a_n = (A^n \mathbf{1})_{s_0} \le \bar\lambda^n V_{s_0}/\min V$. With Lemma 1, $c_n^{1/n} \le (V_{s_0}/\min V)^{1/n} \bar\lambda \to \bar\lambda$. □

No Perron–Frobenius theory, irreducibility or aperiodicity is used; the inequality is checked state by state. (The best $\bar\lambda$ for a given automaton is its spectral radius,

approached when V is close to a Perron vector.) Everything about *how* the automaton was found (state selection, candidate children, optimisation) is irrelevant to soundness: only (1), $F(s_0) = \emptyset$, $V > 0$ and $AV \le \bar{\lambda} V$ matter, and these are what the checker verifies.

4 The automaton actually used

4.1 4.1 States

Fix an even integer K. A state is a self-avoiding walk $w = (v_l = B, \dots, v_1, v_0 = A)$ written relative to its last vertex $A = 0$ (the current position), up to $\mathcal{G}$; its forbidden set is $F(w) = \{v_1, \dots, v_l\}$. It is *admissible* when $l + \delta(w) \le K$, where $\delta(w)$ is the length of a shortest lattice path from A to B avoiding $v_1, \dots, v_{l-1}$ ($\delta = l$ for $l \le 1$): $l + \delta$ is the length of the shortest closed walk that runs through w and returns to B, the Pönitz–Tittmann notion ("the walk can still be the first part of a loop of length $\le K$"). S_K is the set of all admissible walks up to symmetry, plus the empty walk (start state).

Encoding and symmetry reduction. A walk is stored as its steps read from the newest one, $t_0, \dots, t_{l-1}$, $t_i \in \{0, \dots, 2d-1\}$ (direction $2a$ is $+e_a$, $2a+1$ is $-e_a$); positions $p_0 = 0$, $p_{i+1} = p_i - e(t_i)$. The canonical representative of the $\mathcal{G}$-orbit relabels the axes in the order in which they first appear in $t_0, t_1, \dots$ and makes the first step along each axis positive; it is unique in the orbit (two orbit elements with this property differ by a g fixing every axis that occurs, which acts trivially on the walk). Code: 3 bits per step for $d \le 4$, 4 bits for $d = 5, 6$; length in bits 57–62; bit 63 set. The states are enumerated directly (depth-first search from A into the past over canonical sequences, pruned by the monotone bound $j + |p_j|_1 \le K$); δ is computed exactly (monotone-path dynamic programming on the box spanned by A, B, breadth-first search when needed).

4.2 4.2 Children

From state w and direction $e \notin F(w)$, the sites known to be visited, seen from the new position $A' = e$, are $Q = \{p_i - e : 0 \le i \le l\}$. Any self-avoiding path $A' = P_0, P_1, \dots, P_m$ with $P_1, \dots, P_m \in Q$, read as a walk ending at A', has forbidden set $\subseteq (F(w) \cup \{0\}) - e$: its canonical form satisfies (1) and is a valid successor whenever it is a state. Candidates: for every branch of the depth-first search over such paths (cut at $m + |P_m|_1 \le K$), the longest prefix that is a state (tested by looking its canonical code up in the state table); candidates whose site set is strictly contained in another candidate's site set are discarded (this never changed an optimum in our tests). The path that follows w backwards in time gives the Pönitz–Tittmann child (delete the oldest sites until the walk is admissible); the others use *contacts* (non-consecutive sites at distance 1) and keep sites close to the current position while dropping intermediate ones (Couronné's "small bridge" is the case of one contact $v_i \sim v_{i+3}$).

4.3 4.3 Choosing the children

Every choice function $\sigma(s, e) \in C(s, e)$ gives a valid automaton. We minimise the growth rate: with $T(v)_s = \sum_e \min_{t \in C(s,e)} v_t$ (monotone, 1-homogeneous), $K \le 16$ uses min-sum value iteration $v \leftarrow T(v)/\max T(v)$ with all candidates in memory (`src/saw_prod.cpp`); $K = 18$ uses policy iteration with one stored child per (s, e): candidates are computed once, cached on disk, and each pass picks the argmin of the current vector, the initial policy coming from the $K = 16$ vector (`src/saw_big.cpp`). Policy iteration converges in 3–5 passes (tested at $K = 12, 14$: same optimum as value iteration to 10^{-8}). At $K = 18$:

passes 0–5 give 4.72323, 4.72215, 4.72205, 4.72204, 4.72204, 4.72204 (float brackets); the first pass (candidate generation, $6.8 \cdot 10^9$ candidate entries) takes 3 h 04 with 5 threads, the others 1–3 min each.

4.4 4.4 From the vector to the certificate

v is normalised to $\max v = 1$; $V_s = \max(1, \lceil 2^{50} v_s \rceil) \in \mathbb{Z}$ and $\bar{\lambda} = P/10^6$, $P = \lceil 10^6 \max_s (AV)_s / V_s \rceil$, the maximum computed exactly with 128-bit integers.

5 Certificates and how they are checked

Formats (little endian): `SAWCERT1` = magic (8 ASCII bytes), d, K (u32), N, P, Q (u64) — a 40-byte header —, N sorted state codes, N weights V (u64). In a code, step t_i occupies bits $[b\,i,\ b\,i + b)$ with $b = 3$ for $d \le 4$ and $b = 4$ for $d = 5, 6$ (newest step t_0 in the lowest bits; bits above the last step are zero), the length l bits 57–62, and bit 63 is set. `SAWCERT2` = the same followed by the explicit automaton: $N \cdot 2d$ u32 entries, entry $2d\,s + j$ being the index of the chosen child of state s for direction j (same direction code as the steps: $2a \mapsto +e_a$, $2a + 1 \mapsto -e_a$, in the state's own frame); $2^{32} - 1$ = none, used for the forbidden directions, whose entries are ignored. File size $40 + 16N$ ($+8dN$ for `SAWCERT2`). The checker `src/verify_cert.cpp` (written separately from the generators, no shared code) verifies:

- (C0) codes strictly increasing and decodable (at most 57 payload bits, step values $< 2d$); all $1 \le V \le 2^{60}$; the empty walk ($F = \emptyset$) is listed;
- (C1) for every listed s and every $e \in E \backslash F(s)$: a listed child t and a signed permutation g with $gF(t) \subseteq (F(s) \cup \{0\}) - e$, tested point by point. For `SAWCERT2` the child is the listed one; for `SAWCERT1` the checker searches candidates itself (paths through W, as in §4.2) and takes one of minimal weight — the search only finds children, the inclusion test is what is relied on. g is found by enumerating $\mathcal{G}$ ($d \le 3$) or by exhaustive backtracking over partial signed permutations ($d \ge 4$);
- (C2) $Q \sum_e V_{\sigma(s,e)} \le P\,V_s$ for every listed s, in exact 128-bit arithmetic (sums of at most $2d \le 12$ weights $\le 2^{60}$ fit in 64 bits).

(C0)–(C2) are exactly the hypotheses of Lemmas 1–2 with S = the listed states; a PASS proves $\mu_d \le P/Q$.

Further checks (`scripts/check_small.sh`, `scripts/check_counts.py`):

- a second checker in pure Python (`scripts/verify_small_bruteforce.py`) that, for every (s, e), scans *all* listed states and *all* $g \in \mathcal{G}$ (no search, no canonical forms, no hashing); it reproduces the C++ maximal ratio exactly for $K = 8$ (4.771089138921) and $K = 10$ (4.753600356000);
- tamper tests: claimed P lowered by one unit, one weight lowered by 10 %, the one-step state removed, a zero weight, and (SAWCERT2) a child pointer redirected or removed — every corrupted copy is rejected;
- the C++ checker's two search modes (all prefixes / longest prefix per branch) give identical maximal ratios on $K = 8, 10, 12$;
- Pönitz–Tittmann reproduced (child = longest admissible suffix only): $d = 3$: 4.864537, 4.807411, 4.777970, 4.759838, 4.747525, 4.738631 for $K = 4, \ldots, 14$ (published 4.8646, 4.8075, 4.7780, 4.7599, 4.7476, 4.7387); $d = 2$: 2.724799, 2.711252, 2.701374, 2.693849 for $K = 10..16$ (published 2.7248, 2.7113, 2.7014, 2.6939); $d = 4$: 6.891671, 6.851216, 6.830290, 6.817856, 6.809695 ($K = 4..12$; published 6.8917,

6.8513, 6.8303, 6.8179, 6.8097); $d = 5$: 8.910420, 8.881554, 8.867888, 8.860159 ($K = 4..10$; published 8.9105, 8.8816, 8.8679, 8.8602); $d = 6$, $K = 10$: 10.888569 (Noonan's 10.8886). The independent verifier's own implementation of the P–T automaton (`verification/pt_auto.cpp`) reproduces every entry of P–T's Table 2, including the ones not recomputed above: $d = 4$, $k = 14$: 6.8039597 (published 6.8040, with exactly the 4 235 309 states of our $d = 4$, $K = 14$ certificate) and $d = 2$, $k = 4, 6, 8, 18, 20, 22$: 2.8311772, 2.7755911, 2.7444582, 2.6879244, 2.6831389, 2.6791926 (published 2.8312, 2.7756, 2.7445, 2.6880, 2.6832, 2.6792);

- path counts: $a_n \geq c_n$ for $n \leq 24$ in every run (c_n from the Clisby–Liang–Slade tables, = OEIS A001412 for $d = 3$), with equality for $n \leq K - 1$ at least ($n \leq K + 1$ with the Pönitz–Tittmann child).

6 Results

$d = 3$:

K	states	P–T child	optimised (float)	certified $\bar{\lambda}$	generation	independent check
10	2 723	4.7598376	4.7536004	4.753601	<1 s	C++ + Python PASS
12	41 425	4.7475251	4.7417726	4.741773	2 s	C++ PASS
14	678 730	4.7386308	4.7333265	4.733327	30 s	C++ PASS
16	11 641 894	4.7319086	4.7269837	**4.726984**	7 min (6 thr.)	C++ PASS (13 min)
18	206 118 560	—	4.7220473	**4.722048**	3 h 25 (5 thr., 10 GB)	C++ PASS on SAW-CERT2 (4 min 47 s, 8 GB)

$d = 4, 5, 6$ (one thread; every certificate in the table was re-checked by `verify_cert` — d = 4, K = 14 in 15 min, d = 5, K = 14 in 32 min, the others in under 30 s; logs `certificates/d*_k*.verify.log`):

d	K	states	P–T child	certified $\bar{\lambda}$	previous record
4	12	139 267	6.8096952	6.806500	
4	14	4 235 309	6.8039597 (verifier)	**6.801078**	6.8040
5	10	5 732	8.8601588	8.858371	8.8602
5	12	187 011		8.853752	
5	14	7 787 256		**8.850674**	
6	10	5 732	10.8885689	10.887669	10.8886
6	12	192 224		**10.885008**	

7 Limitations and remarks

- The gain over Pönitz–Tittmann comes from the choice of children, not from more memory: at equal K it is 0.0057 (K = 12), 0.0053 (K = 14), 0.0049 (K = 16) in $d = 3$; 0.0032 (K = 12) in $d = 4$.
- Couronné's other devices for $\mathbb{Z}^2$ (large bridges, planar arguments, "+2" loop budget for line-like states or states lacking simplifications) are planar or, in our $d = 3$ tests, worse than increasing K. Refining only the states of large stationary mass (hybrid $S_{K-2}\cup$ part of S_K) was also tested and is not efficient.
- Pönitz–Tittmann's abstract prints 2.679192495 for $d = 2$, $k = 22$; vitplanocka (GitHub, 2026) certify the automaton constant in $[2.679192641375, 2.679192650414]$. Our P–T-rule automaton ($d = 2$, $K = 22$, 8 880 776 states) gives the floating bracket $[2.6791926494, 2.6791926495]$, confirming that erratum; the four-decimal entries of their Table 2, including $d = 3$: 4.7387, are correct.
- Implementation remark: for $d = 2$ the floating Perron vector underflows on dead-end states (walks trapped in a pocket), and the rounding $V = \max(1, \lceil 2^{50} v \rceil)$ then gives a useless ratio (3) — a repair sweep would be needed. This does not occur for $d \geq 3$: every certificate listed here passes with a maximal ratio equal to the floating eigenvalue to 10 digits.
- Heuristic extrapolation (not a proof): successive gains 0.0118, 0.0084, 0.0063 for K = 10→12→14→16 suggest ≈4.722 at K = 18 — obtained: 4.722048. $K = 20$ would need $\approx 3.5 \cdot 10^9$ states, beyond 14 GB with this design.
- For $d \geq 5$ the "previous record" column relies on CLS 2007 §7.2 and our searches; the computer-assisted lace-expansion papers (Hara–Slade 1992) were not read; Slade's 1995 survey (Table 1) lists only Alm's upper bounds (4.756, 6.832, 8.881, 10.903 for d = 3..6), the lace expansion being used there for lower bounds. The independent verifier read Slade 1995 and CLS 2007 §7.2 directly (Slade is a co-author of both and lists no lace-expansion upper bound) and repeated the novelty search (`verification.md` §6); the $d = 6$ value 10.8886 is attributed to Noonan 1998 because CLS cite [Noonan, P–T] jointly and P–T have no $d = 6$ column (Noonan's paper itself was not read).

8 What an independent verifier should examine

1. Lemmas 1–2 (frames: g acts linearly, positions relative to the current vertex).
2. That `src/verify_cert.cpp` checks exactly (C0)–(C2): decoding $p_{i+1} = p_i - e(t_i)$; forbidden set $\{p_1, \ldots, p_l\}$ (current vertex excluded); only steps $e \in F(s)$ are skipped; $W = \{p_i - e : 0 \leq i \leq l\}$; the inclusion test (enumeration of $\mathcal{G}$ for $d \leq 3$, backtracking for $d \geq 4$); overflow.
3. Generators and checker were written by the same agent (separately). The Python brute-force checker is logically independent but only practical for $K \leq 10$. Re-implementing (C0)–(C2) (~150 lines) and running it on the stored certificates is the strongest test; `SAWCERT2` makes this a purely local check.
4. Large certificates are not in the repository ($d = 3$: K = 16, 186 MB; K = 18, 3.3/8.2 GB; $d = 4$ K = 14, 68 MB; $d = 5$ K = 14, 125 MB): SHA-256 in `certificates/SHA256SUMS.txt`, deterministic regeneration scripts.
5. Novelty for $d = 4, 5, 6$ (see §7).

Done: see `verification.md` (independent checker `verification/sawcheck.cpp`, written

from §3 and §5 only; all certificates PASS with the same exact maximal ratios; tamper tests on both checkers; $a_n \geq c_n$ cross-check; independent reproduction of P–T's Table 2; novelty search). Verdict: CONFIRMED WITH CORRECTIONS (editorial corrections only; no bound changed).